



xWRL6844

mmWave Demo Parameter Tuning and Customization Guide

Version 2.0

Sept 2025



Texas Instruments, Incorporated
12500 TI Boulevard
Dallas, Texas 75243 USA

Version History

Version	Date	Comment
1.0	March 2025	Initial version.
2.0	Oct 2025	Updates to Timing Requirements

Table of Contents

1	Introduction and Scope.....	4
2	Signal Processing Algorithm.....	5
2.1	Overall Signal Processing Flow	5
2.1.1	MIMO Demodulation	5
3	Configuration Parameters.....	7
3.1	Sensor Front-End Parameters	9
3.1.1	Chirp Profile Configuration.....	9
3.1.2	Frame Configuration	14
3.1.3	APLL Frequency Shift.....	15
3.1.4	Low-Power Mode Configuration	16
3.1.5	Factory Calibration Configuration	16
3.1.6	Summary of Timing Constraints	18
3.2	Detection Layer Parameters	19
3.2.1	Processing Chain Configuration	19
3.2.2	Peak Detection Configuration	19
3.2.3	Region of Interest Configuration.....	22
3.2.4	Clutter Removal Configuration	24
3.2.5	Antenna Pattern Configuration.....	24
3.2.6	Range Bias and Phase Compensation Configuration	26
3.2.7	GUI Monitoring Configuration	29
3.3	Debug Related Parameters.....	30
4	Parameter Tuning and Performance.....	32
4.1	Effect of the Radar Parameters on the Physical Requirements	32
4.2	How to Configure the Range	32
4.3	How to Configure the Motion Sensitivity	34
4.4	How to Configure the Angular Accuracy	35
5	References	36

1 Introduction and Scope

This document provides a high-level overview of the signal processing chain of the xWRL6844 mmWave demo and presents a guide to tune the demo for different use cases and environments. It explains the configurable processing chain parameters available. These parameters may need to be based on the end equipment needs to improve the overall demo performance. It is important to note that the scope of this document is limited to introducing the users to available tuning parameters, their definitions and boundaries for the values that are acceptable for the demo. The implementation details of the processing chain, including the execution flow, memory management, etc., are given in a separate demo implementation guide. For the implementation details of the demo please refer to `*SDK_INSTALL_PATH/README_FIRST_xWRL684x.html/Examples/MmwaveDemos/*` in SDK [1].

Section 2 briefly introduces the overall signal processing chain. Section 3 details the command-line interface (CLI) of the device, with all the parameters that can be tuned. Section 4 describes common examples in which the user may tune the CLI parameters to improve performance.

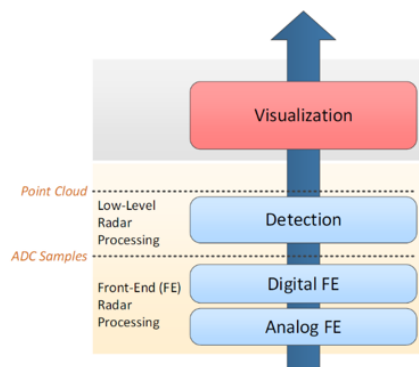


Figure 1. The signal processing layers of the mmWave demo.

The xWRL6844 mmWave demo has several layers, as illustrated in the flow diagram in Figure 1.

At the lowest level, the front-end radar processing layer handles the radar signaling parameters such as chirp, frame, modulation type, sampling rate etc. Afterward, the detection layer takes the analog-to-digital converter (ADC) data for the IF (intermediate Frequency/beat) signal from the sensor front-end as input. The detection layer is responsible for sensing multiple reflections from the three-dimensional (3D) environment around the sensor and delivering a set of measurement vectors, known as the point cloud, representing the real-life targets in the scene. Each measurement vector generated by the detection layer represents a reflection point with range, azimuth, elevation, radial velocity (i.e., Doppler), and signal-to-noise ratio (SNR).

As depicted in Figure 1, after the detection layer has been completed, the processing chain on the device provides the point cloud data and other detection layer data (heatmaps, etc.) to the host. In the current version of the demo, only the data visualization to the user is implemented on the host in a Graphical User

Interface (GUI) based application. As illustrated in Figure 1, all other processing blocks are running on the device in real-time.

2 Signal Processing Algorithm

This chapter will summarize the high-level signal processing algorithm of the mmWave demo.

2.1 Overall Signal Processing Flow

The chirp processing module running on the Hardware Accelerator (HWA) takes the ADC samples per chirp from the front end for all the physical receive channels (antennas), performs the range Fast Fourier Transform (FFT), then stores the output to the radar cube. The radar cube data is then processed one range bin at a time. For each range gate, phase/gain calibration for all the antennas is applied. Doppler and angle processing across all chirps and antennas within the frame is handled by the HWA, which is configured by the R5F core based on CLI parameters such as FFT sizes, azimuth and elevation field of view, etc.

After the range gate processing, the detection matrix (i.e., heatmap) is created. The HWA performs the peak detection processing on the generated range-doppler heatmap with some operations assisted by the R5F core to create the output list and other attributes of the detected peaks. The final peak detection list is then shipped to the host to feed the higher processing layers. The current peak detection processing is performed in the range-doppler domain.

2.1.1 MIMO Demodulation

To achieve full MIMO capabilities, the xWRL6844 demo uses one of three demodulation techniques: 2TX_BPM-MIMO, 4TX_BPM-MIMO, and TDM-MIMO. Figure 2 below depicts both TDM-MIMO with 2 TX antennas and 2TX_BPM-MIMO schemes, and Figure 3 depicts TDM-MIMO with 4 TX antennas and 4TX_BPM-MIMO. The following sections briefly summarize the modulation schemes.

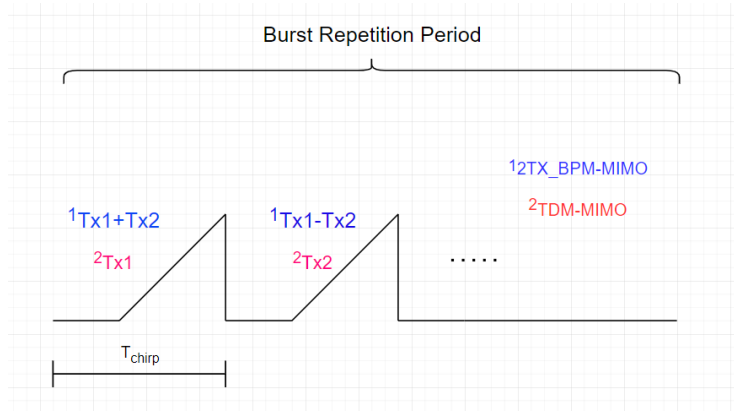


Figure 2: Chirp timing diagram 2TX_TDM-MIMO and 2TX_BPM-MIMO.

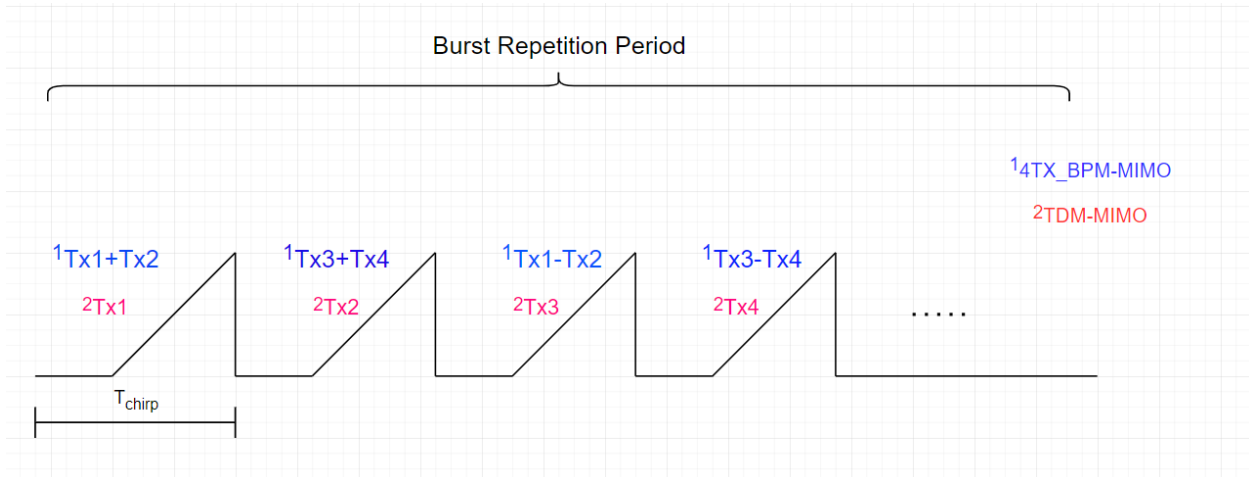


Figure 3: Chirp timing diagram 4TX_TDM-MIMO and 4TX_BPM-MIMO.

2.1.1.1 TDM-MIMO

In the TDM-MIMO scheme, only one TX antenna is active to transmit a chirp. All the TX antennas are then enabled in the frame in an alternating manner. For example, for 4 TX (TX1, TX2, TX3, and TX4), chirp 0 has only TX1 enabled, chirp 1 has only TX2 enabled, chirp 2 has only TX3 enabled, chirp 3 has only TX4 enabled, chirp 4 has only TX1 enabled, and so on. Therefore, no additional demodulation step is needed in the TDM-MIMO scheme because the chirps from the TX antennas are separated in the time domain and stored in the radar cube properly.

2.1.1.2 BPM-MIMO

In the BPM-MIMO scheme, a frame consists of multiple blocks. However, unlike in TDM-MIMO, where only one TX antenna is active per chirp interval, two transmit antennas are active in each chirp interval. For the 2TX_BPM-MIMO depicted in Figure 2, in each time slot, transmit antennas are configured to transmit with the following phases, where (+) denotes positive phase and (-) denotes negative phase:

Chirp Interval	TX1	TX2
1	(+)	(+)
2	(+)	(-)

For the 4TX_BPM-MIMO depicted in Figure 3, in each time slot, transmit antennas are configured to transmit with the following phases, where (+) denotes positive phase and (-) denotes negative phase:

Chirp Interval	TX1	TX2	TX3	TX4
1	(+)	(+)		
2			(+)	(+)
3	(+)	(-)		
4			(+)	(-)

Therefore, when the BPM mode is enabled, the signal processing chain must decode the received chirp pairs to separate the transmission from each TX antenna.

2.1.1.3 BPM Decoding

Let TX1, TX2, TX3, and TX4 represent chirp signals from four TX antennas in 4TX_BPM-MIMO. As illustrated in Figure 3, in each time slot, the following signal combinations are transmitted:

- SA= TX1+TX2
- SB= TX3+TX4
- SC= TX1-TX2
- SD= TX3-TX4

Using the corresponding received signals (SA, SB, SC, and SD) at a specific received RX antenna, the components from the individual transmitters are separated using:

- TX1= (SA+SC)/2
- TX2= (SA-SC)/2
- TX3= (SB+SD)/2
- TX4= (SB-SD)/2

Similarly, 2TX_BPM-MIMO is decoded utilizing the corresponding received signals.

With simultaneous transmission on both TX antennas, the total transmitted power per chirp interval is increased, and it can be shown that this translates to an SNR improvement.

3 Configuration Parameters

The parameters presented in this section are used to adjust the radar's performance to meet a variety of requirements and use-cases. The user can set these parameters using the configuration (i.e., cfg) files located in the demo application folders. An example configuration file is given in Table 1. Each line in these files represents a CLI message that configures several parameters. It is important to note that Table 1 may not have the correct sequence of the commands that the demo requires. For the correct sequence, please refer to the cfg files located in the demo application folders and the subsequent sections.

Table 1. The commands in an example configuration (cfg) file for the xwrL6844 mmWave Demo.

No	Parameter group	CLI commands and arguments
1	Sensor front-end parameters	sensorStop 0 channelCfg 153 255 0 chirpComnCf 8 0 0 256 1 13.1 3 chirpTimingCf 6 63 0 160 58 adcDataDitherCf 1 frameCfg 64 0 1358 1 100 0 lowPowerCfg 1

		factoryCalibCfg 1 0 44 2 0x1ff000 runtimeCalibCfg 1 sensorStart 0 0 0 0
2	Detection layer parameters	guiMonitor 1 1 0 0 0 1 aoaProcCfg 64 64 cfarProcCfg 0 2 8 4 3 0 9.0 0 cfarProcCfg 1 2 4 2 2 1 9.0 0 cfarFovCfg 0 0.25 9.0 cfarFovCfg 1 -20.16 20.16 aoaFovCfg -60 60 -60 60 clutterRemoval 1 antGeometryBoard xWRL6844EVM
3	Debug related parameters	adcDataSource 0 adc_data_name.bin adcLogging 0 gpAdcMeasConfig 0 0

There are some additional parameters (and the configuration options they provide) not detailed in this document (at the sensor front-end level), and they are described in the firmware folder in SDK [1]. In this document, we also briefly review these parameters and discuss their effect and limitations in the overall mmWave demo. TI strongly recommends using the provided values of the configuration parameters as the best empirical values obtained from testing in different environments. The following sections provide detailed information on each parameter and discuss their effect on the detection performance for different scenarios.

The first table below summarizes the default commands, which do not need a specific tuning, to configure the RF module parameters of the radar sensor, such as enabling the transmit and receive antennas, start and stop the sensor, etc. These parameters are recommended for achieving optimal performance.

Commands	Parameters	Usage in demo	Notes
sensorStop	reserved	Set to 0	This command stops the sensor from transmitting further frames and is not supported when lowPowerCfg is 1 (see the following sections). The recommended alternative is to power cycle for a reconfiguration.
channelCfg	rxChCtrlBitMask	Set to 153 to enable all available RX antennas.	To enable (as an example): - RX antennas 1, 2, 3 and 4, mask = 0x10011001b = 153 - RX antennas 1 and 2, mask = 0x00001001b = 9 - RX antennas 2 and 3, mask = 0x00011000b = 24 - RX antennas 1, 2 & 3, mask = 0x00011001b = 25
	txChCtrlBitMask	Set to 255 to enable all available TX antennas.	To enable (as an example): - TX antennas 1, 2, 3 and 4, mask = 0x11111111b = 255 - TX antenna 1, 2, and 3, mask = 0x1111111b = 63 - TX antenna 2, and 3, mask = 0x111100b = 60
	reserved	Set to 0	Reserved
sensorStart	frameTrigMode	Set to 0	Always set to 0 in the current version of the SDK to enable frame SW immediate trigger mode.
	chirpStartSigLbEn	Set to 0	Always set to 0 in the current version of the SDK.
	frameLivMonEn	Set to 0	Reserved
	frameTrigTimerVal	Set to 0	Always set to 0 in the current version of the SDK.

3.1 Sensor Front-End Parameters

The sensor front-end parameters, which configure the mmWave demo at the frequency modulated continuous wave (FMCW) signal level, are given in this section. The following subsections briefly summarize these parameters. To get a general understanding of the sensor front-end parameters, please refer `*SDK_INSTALL_PATH/firmware/mmwave_dfp/docs/mmwave_dfp_interface_control_document*` in SDK [1].

3.1.1 Chirp Profile Configuration

The xWRL6844 transmits and receives FMCW radar waves. The basic principles of FMCW radars are well-reported in the literature [3][4]. The [chirpComnCf](#) and [chirpTimingCf](#) commands let the user modify the FMCW chirp waveform on the xWRL6844. This section summarizes these commands and the parameters they use. The parameters given in these commands are common for all chirps in a frame.

Note that the tables given in the rest of the document present the data type and description of each parameter along with the suggested values in an example scenario. The data types are only classified as 'int' for integer numbers and 'float' for floating-point numbers. This document does not cover the actual byte size of the parameters (uint8, int32, etc.). Instead, the supported values or valid ranges in each parameter are given in the corresponding section. For more details about the actual data size, please refer to `*SDK_INSTALL_PATH/firmware/mmwave_dfp/docs/mmwave_dfp_interface_control_document*` in SDK [1].

chirpComnCf 8 0 0 256 1 13.1 3			
Parameter	Type	Value	Description
digOutputSampRate	int	8	Sampling rate decimator. The digital sampling rate is given by $200\text{MHz}/\text{digOutputSampRate}$. Valid Range: 8 to 100. Supported digOutputSampRate: 8, 10, 13, 15, 20, 25, 40, 50, 60, 80, 100. Refer to [3] for the details about the sampling rate.
digOutputBitsSel	int	0	Digital output sample bits select. Set this field to 0 if chirp accumulation feature is used (refer numOfChirpsAccum field in frameCf) Refer to the note below ¹ for the details.
dfeFirSel	int	0	The final stage FIR filter's characteristics. Refer to the note below ² for the details.
numOfAdcSamples	int	256	Number of ADC samples collected during the ADC sampling time. Only values that are powers of 2 are supported. Valid Range: 2 to 1024. Refer to [3] for the details.
chirpTxMimoPatSel	int	1	Configure BPM/TDM mode. Set to 1 to enable TDM mode Set to 4 to enable BPM mode. NOTE: 2TX or 4TX BPM demodulation is performed based on TX channels selected in channelCf cmd. Refer to the note below ³ for the details.
chirpRampEndTime	float	13.1	Ramp end time in μs . Refer to [3] and the description below ⁴ for the details.
chirpRxHpfSel	int	3	Chirp profile HPF corner frequency. Refer to the note below ⁵ for the details.

¹[digOutputBitsSel](#): This field controls the section of bits from the internal 16-bit output, which will be stored in the ADC buffer. The Rx ADC always sends 16-bits to the output. But, depending on the

configuration up to 4-bits are dropped from the result, and the result is sign extended to map to a valid signed 16-bit number. xWRL68xx devices support the following output options:

- 0 - Digital sample output is 12 MSB bits of DFE after rounding 4 LSBs.
- 1 - Digital sample output is 12 bits after rounding 3 LSBs & clipping 1 MSB.
- 2 - Digital sample output is 12 bits after rounding 2 LSBs & clipping 2 MSB.
- 3 - Digital sample output is 12 bits after rounding 1 LSBs & clipping 3 MSB.
- 4 - Digital sample output is 12 LSB bits after clipping 4 MSB.
- 5 - Digital sample output is 16 bits.

When `numOfChirpsAccum` is set to values between 2 and 16, the recommended setting of `digOutputBitsSel` is 0. When `numOfChirpsAccum` is set to 0, the recommended setting of `digOutputBitsSel` is 5. Other configurations may lead to saturation and should generally be avoided.

²dfeFirSel: This option selects the configuration of the final DFE FIR filter. The options are as follows:

- 0 - Long Filter (90% visibility): This provides visibility to a larger range of IF frequencies: 0 to 0.45 x Sampling Rate. Beyond that, the filter response starts dropping and enters the filter transition band.
- 1 - Short Filter (80% visibility): This provides faster settled outputs, but the IF frequency range visible is 0 to 0.40 x Sampling Rate. Beyond that, the filter response starts dropping and enters filter transition band.

³chirpTxMimoPatSel: This parameter configures the TX antenna multiplexing scheme (BPM or TDM) to enable the MIMO mode on the sensors. For more information, refer to Section 2.1.1.

⁴chirpRampEndTime: This parameter is the sum of the ADC start time (refer to `chirpAdcSkipSamples` parameter below), ADC sampling time, and some ramp excess time. The user should program Ramp End Time higher than the time necessary for RX ADC sampling and filtering. The time necessary is a function of the sampling rate, F_s , in Hz, determined by `digOutputSampRate`, as well as the number of skip samples (`chirpAdcSkipSamples`) and collect samples (`numOfAdcSamples`). Specifically,

$$\text{Ramp End Time} \geq (\text{Skip Samples} + \text{Collect Samples} - 3) / F_s$$

In setting the chirp timing parameters and sample counts, the user should ensure that the programmed chirp periodicity, i.e., Ramp End Time + Idle Time (set by fields: `chirpRampEndTime`, `chirpIdleTime`) always exceeds the time necessary for RX ADC sampling. Specifically, ensure that

$$(\text{Ramp End Time} + \text{Idle Time}) \geq (\text{Skip Samples} + \text{Collect Samples} + 4) / F_s$$

Also, `chirpRampEndTime` must be $\geq \frac{(\text{chirpAdcSkipSamples} + \text{numOfAdcSamples} - 3)}{\text{Samplingrate (Msps)}}$.

⁵chirpRxHpfSel: Chirp profile HPF corner frequency. Increased HPF cutoff frequency helps to avoid ADC saturation when there is high coupling from TX to RX antennas. The supported options are given below:

- 0 - 175kHz HPF corner frequency

- 1 - 350kHz HPF corner frequency
- 2 - 700kHz HPF corner frequency
- 3 - 1400kHz HPF corner frequency

chirpTimingCfg 6 63 0 160 58			
Parameter	Type	Value	Description
chirpIdleTime	float	6	Idle time in μ s. Refer to [3] and the note below ¹ for the details.
chirpAdcSkipSamples	int	63	Number of samples skipped during ADC sampling. The ADC start time in μ s is computed based on the sampling rate. Valid range: 0 to 2000/ digOutputSampRate . Refer to [3] and the note below ² for the details.
chirpTxStartTime	float	0	TX start time in μ s. Valid Range ≥ 0 Recommended Value: 1.5 μ s (to avoid EIRP peaking at start frequency) Refer to [3] and the note below ¹ for the details.
chirpRfFreqSlope	float	160	Frequency slope in MHz/ μ s. Valid range: - 399MHz/ μ s to +399MHz/ μ s. Refer to [3] for the details.
chirpRfFreqStart	float	58	Chirp profile RF start frequency in GHz. Valid range: 57GHz to 64GHz Refer to [3] and the note below ³ for the details.

¹**chirpIdleTime and chirpTxStartTime:** The parameter [chirpIdleTime](#) indicates the idle time in the chirp cycle before starting of the frequency ramp. Along with the ramp end time, the number of TX antennas, and the number of chirp loops in a frame (as discussed in Section 3.1.2), this parameter configures the motion sensitivity (i.e., velocity resolution) of the detection layer. The user can adjust this parameter according to desired motion sensitivity. It is important to note that the user must ensure that the total chirping time and the required processing time can fit into a single frame period, which is discussed in Section 3.1.2. The parameter [chirpTxStartTime](#) indicates the TX start time in the chirp cycle with respect to the knee of the ramp.

- If [chirpIdleTime](#) is $\geq$ maximum of $\{6\mu\text{s} - \text{chirpTxStartTime}, 3.1\mu\text{s}\}$ then the SDK code enables inter-chirp power save for TX and RX chains.
- Else-if [chirpIdleTime](#) is $\geq$ the maximum of $\{4.1\mu\text{s} - \text{chirpTxStartTime}, 3.1\mu\text{s}\}$ then the inter-chirp power save mode is enabled for TX chains but disabled for RX chains.
- Else, it will throw an error. Therefore, the [chirpIdleTime](#) shouldn't be configured lower than maximum of $\{4.1\mu\text{s} - \text{chirpTxStartTime}, 3.1\mu\text{s}\}$.

Also,

$$\text{chirpIdleTime} \geq 1.21\mu\text{s} + \frac{\text{chirpRfFreqSlope}}{1400 \text{ MHz}/\mu\text{s}} \times \text{chirpRampEndTime}$$

²chirpAdcSkipSamples: The parameter configures the number of ADC samples to be skipped at the knee of the ramp, which controls the ADC sampling start time. Note that the term “skip samples” refers to the number of ADC samples at the beginning of each chirp which are skipped (i.e. not collected for radar processing). In each chirp, the initial samples exhibit settling behavior, which arises in the synthesizer PLL and RX baseband high-pass and low-pass filters (analog and digital). Figure below describes various chirp configuration parameters.

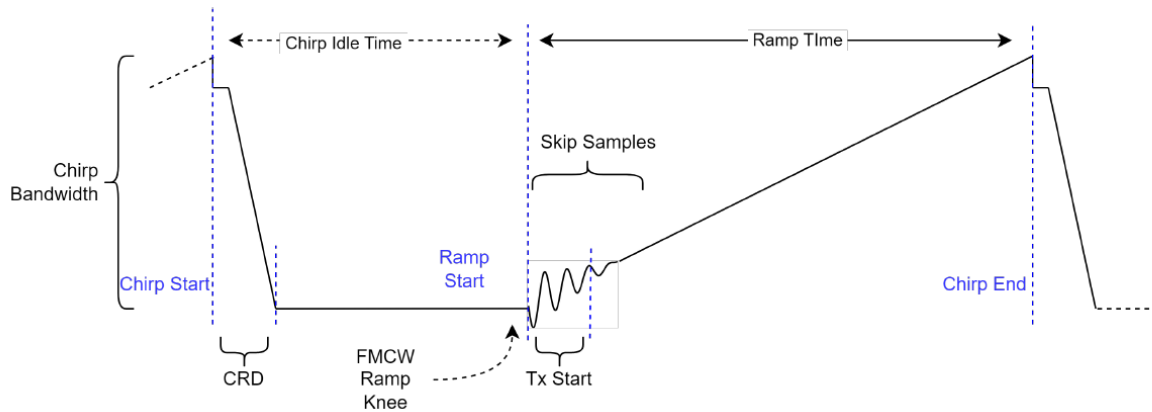


Figure 4: FMCW Chirp Parameters

The skip samples parameter needs to be high enough to wait out these (potentially abrupt) initial transients.

- Longer settling is needed if chirp starts with <57.5GHz.
- Also, settling time is dependent on the filter choice
 - Long filter requires longer settling time but allows higher IF bandwidth (0.45xSampling rate) which is 90% of Nyquist
 - Short filter requires lesser settling time but allows slightly reduced IF bandwidth (0.4xSampling rate) which is 80% of Nyquist

Specifically, the recommended parameters are given as:

Chirp start freq. (60GHz devices)	dfeFirSel	Skip Samples Recommended
[57.5, 64] GHz	0 (long filter)	$\text{Num Skip Samples} \geq 12 + \max[(\text{chirpTxStartTime} + 1.5\mu\text{s}) * F_s + 4, (2\mu\text{s} * F_s)] \text{ samples}$
	1 (short filter)	$\text{Num Skip Samples} \geq 08 + \max[(\text{chirpTxStartTime} + 1.5\mu\text{s}) * F_s + 3, (2\mu\text{s} * F_s)] \text{ samples}$
Otherwise	0 (long filter)	$\text{Num Skip Samples} \geq 12 + \max[(\text{chirpTxStartTime} + 1.5\mu\text{s}) * F_s + 4, (3\mu\text{s} * F_s)] \text{ samples}$
	1 (short filter)	$\text{Num Skip Samples} \geq 08 + \max[(\text{chirpTxStartTime} + 1.5\mu\text{s}) * F_s + 3, (3\mu\text{s} * F_s)] \text{ samples}$

Note: the 1.5μs in the above table corresponds to the recommended HPF Fast Init Duration. The other parameters correspond to the group delay and settling delay of LPFs (which depend on the user’s filter type selection) and synthesizer PLL settling delay (which depends on RF start frequency (see [chirpRfFreqStart](#))).

³chirpRfFreqStart: This parameter is the start frequency of the total occupied bandwidth. Along with the frequency slope and the ADC start/sampling time, this parameter configures the valid bandwidth occupied in the raw FMCW beat signal to be utilized in the detection layer. The default values for the mmWave

demo are configured for the best performance based off the radiation patterns of the xWRL6844 EVM antennas. The user might adjust these values for a customized antenna design.

adcDataDitherCfg 1			
Parameter	Type	Value	Description
adcDataDithEnable	int	1	If enabled, random delay in the range of (1.1us - 3.6us) is added every chirp for the ping-pong switch and interrupt generation w.r.t ADC Valid pulse. 0: Disable 1: Enable

3.1.2 Frame Configuration

The frameCfg command configures the FMCW radar frame timing parameters, which include the number of chirps, burst and frame counts, burst & frame periodicity, etc. For details about a typical frame structure please refer to *SDK_INSTALL_PATH/README_FIRST_xWRL684x.html/Examples/MmwaveDemos/* in SDK [1].

frameCfg 64 0 1358 1 100 0			
Parameter	Type	Value	Description
numOfChirpsInBurst	int	64	Number of chirps in a burst. Valid Range: 1 to 65535 chirps. Refer to the note below ¹ for the details.
numOfChirpsAccum	int	0	Number of accumulations per chirp. 0: Chirp accumulation feature is disabled. 2 – 16: Chirp accumulation feature is enabled. Refer to the note below ² for the details.
burstPeriodicity	float	1358	Burst periodicity in μ s. Refer to the note below ¹ for the details.
numOfBurstsInFrame	int	1	Number of bursts per frame. Refer to the note below ¹ for the details.
framePeriodicity	float	100	Frame periodicity (i.e., the time gap between successive frame starts) in ms. Refer to the note below ³ for the details.
numOfFrames	int	0	Number of total frames. 0: Infinite frames, value > 0: Finite frames. Valid Range: 0 to 65535. Always set to 0 to run the demo in continuous mode (i.e., to generate endless frames).

¹**numOfChirpsInBurst, burstPeriodicity, and numOfBurstsInFrame:** These parameters configure the total number of chirps transmitted within a single frame. Along with the chirp timing parameters in Section 0,

these parameters configure the motion sensitivity (i.e., velocity resolution) of the detection layer and the maximum detection range based on SNR (please refer to Section 4 for details). These parameters affect the radar cube size, required processing time, and average power consumption, which should be carefully optimized for the desired use case.

Although the digital front-end has the flexibility of configuring any combination of chirps/bursts, the mmWave demo only supports a total of 1024 chirps ($\text{NumOfChirpsInBurst} * \text{NumOfBurstsInFrame}$) when all 4 Rx antennas are enabled.

Below snippets from ICD cover the burst and frame time constraints.

Variable	Description	xWRL6844 xWRL6888
Burst Start Time <i>BurstStartTime</i>	Time required for RFS firmware to process the Burst Start event	100 us
Burst End Time <i>BurstEndTime</i>	Time required for RFS firmware to process the Burst End event	30 us
Minimum Burst Idle Time <i>MinBurstIdleTime</i>	$BurstStartTime + BurstEndTime$	135us

Variable	Description	xWRL6844 xWRL6888
Frame Start Time <i>FrameStartTime</i>	Time required for RFS firmware to process the Frame Start event	37 us
Frame End Time <i>FrameEndTime</i>	Time required for RFS firmware to process the Frame End event	17 us
Minimum Frame Idle Time <i>MinFrameIdleTime</i>	$FrameStartTime + FrameEndTime$	54 us

²numOfChirpsAccum: This field indicates the number of chirps to be accumulated before sending the ADC data out in DFE. This feature can be used to increase the SNR without increasing the number of chirps to process in the DSP/HW accelerator. When this mode is enabled, the front-end device will send more chirps than in the processing layer (since they are accumulated before filling the ADC buffer), which will increase power consumption. Hence, this parameter should be carefully optimized for the desired use case and power consumption. When this parameter is set to ≥ 2 , then **digOutputBitsSel** is recommended to be set to 0 for avoiding code saturation after accumulation.

³framePeriodicity: This parameter configures the frame rate of the processing chain. The user must always ensure that the total chirping time and the required processing time can fit into a single frame period. This parameter also affects the average power consumption.

3.1.3 APLL Frequency Shift

apllFreqShiftEn 0		
Type	Value	Description
bool	0	Enable/disable the APLL frequency shift. When enabled, the APLL frequency is configured to 396MHz, and when disabled APLL frequency is configured to 400MHz. 0 - Disable APLL shift. 1 - Enable APLL shift. NOTE: This is an optional command, by default <code>apllFreqShiftEn</code> is set to 0.

APLL frequency shift should be set to 1 when the application targets class 5 CISPR25 with dutycycle > 25%. This is a workaround for APLL Frequency Shift mentioned in errata.

When APLL frequency shift is enabled, all timings except frame period are higher than user programmed values by a factor of (100/99). The timings affected include ADC sampling period, Ramp end time, idle time and burst period.

3.1.4 Low-Power Mode Configuration

The following command enables/disables the low-power mode. This command is used to enable/disable various sensor inter-frame dynamic power save options. For more details please refer to *SDK_INSTALL_PATH/README_FIRST_xWRL684x.html/Examples/MmwaveDemos/* in SDK [1].

lowPowerCfg 1			
Parameter	Type	Value	Description
enable	int	1	Low power framework enable/disable flag: 0: Disabled. 1: Enabled.

3.1.5 Factory Calibration Configuration

The factoryCalibCfg lets the user either save the factory RF calibration performed by the FECSS onto the FLASH or to restore the previously saved RF calibration data from the FLASH and instruct FECSS not to re-perform the factory calibration. Users can either save or restore or perform neither operation. The user is not allowed to save and restore in a given boot sequence simultaneously. **NOTE: Factory Calibration CLI command should be issued within 25°C +/- 15°C.**

factoryCalibCfg 1 0 44 2 0x1ff000			
Parameter	Type	Value	Description
saveEnable	int	1	Save calibration enable/disable flag: 0: Save disabled. 1: Save enabled. Refer to the note below ¹ for the details.

restoreEnable	int	0	Restore calibration enable/disable flag: 0: Restore disabled. 1: Restore enabled. Refer to the note below ² for the details.
rxGain	int	44	RX gain in dB scale. Recommended Range: Even values from 38dB to 46dB.
txBackoff	int	2	TX channel power calibration. Valid Range: 0dB to 20dB. Refer to the note below ³ for the details.
flashOffset	int	0x1ff000	Always set to 0x1ff000. Refer to the note below ⁴ for the details.
monitorsFlashOffset	Int	0x1fe000	Monitors flash address offset. NOTE: This is an optional command

¹saveEnable: When this option is enabled application will boot-up normally and configure the FECSS to perform all applicable factory calibrations during FECSS initialization. Once the calibrations are performed, the application will retrieve the calibration data from FECSS and save it to FLASH. Users need to specify a valid **flashOffset** value. Besides, **restoreEnable** option should be set to 0.

NOTE: When monitors are enabled, the result data (RX loopback gain/phase mismatches with respect to TX0, return loss from all TX Ballbreak monitors, and TX power values) is stored in flash at the location specified by **monitorsFlashOffset**, provided **saveEnable** = 1 and **restoreEnable** = 0.

²restoreEnable: When the restore enabled option is set, the application will check the FLASH for a valid calibration data section. If present, it will restore the data from FLASH and provide it to FECSS while configuring it to skip any real-time factory calibrations and use provided calibration data. Users need to specify the valid **flashOffset** value that was used during the saving of calibration data. Besides, **saveEnable** option should be set to 0. The **rxGain** and **txBackoff** arguments will be ignored when the restore option is enabled.

NOTE: When monitors are enabled, the application will check the FLASH for a valid monitor result data section. If present, it will restore the data from FLASH. Users need to specify the valid **monitorsFlashOffset** value that was used during the saving of monitors data. Besides, **saveEnable** option should be set to 0.

	saveEnable	restoreEnable
Time-0 per-sensor calibration in customer factory (interference free environment at 25C +/- 15C)	1	0
Field operation	0	1

³txBackoff: This parameter is the common output power backoff (in dB scale) for all TX antennas. The backoff value in this field defines how much the transmit power for each transmit antenna should be reduced from the maximum in dB scale. (NOTE: The device can adjust txBackoff in 0.5 dB steps, but the SDK doesn't support this feature).

⁴**flashOffset:** Address offset in the flash to be used while saving or restoring calibration data. Make sure that the reserved sectors in flash are not used. This field is don't care if both save and restore are disabled.

runtimeCalibCfg 1			
Parameter	Type	Value	Description
enable	bool	1	CLPC Enable/Disable: 0: Disabled. 1: Enabled.

3.1.6 Summary of Timing Constraints

<i>chirpTxStartTime</i>	$\geq 0\text{us}$ <i>Recommended value = 1.5us (to avoid EIRP peaking at start frequency)</i>															
<i>chirpAdcSkipSamples</i>	<table><tr><td>Chirp start freq. (60GHz devices)</td><td>c_DfeFirSel</td><td>Skip Samples Recommended</td></tr><tr><td rowspan="2">[57.5, 64] GHz</td><td>0 (long filter)</td><td>Num Skip Samples $\geq 12 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 4, (2\text{us} * F_s)]$ samples</td></tr><tr><td>1 (short filter)</td><td>Num Skip Samples $\geq 08 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 3, (2\text{us} * F_s)]$ samples</td></tr><tr><td rowspan="2">Otherwise</td><td>0 (long filter)</td><td>Num Skip Samples $\geq 12 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 4, (3\text{us} * F_s)]$ samples</td></tr><tr><td>1 (short filter)</td><td>Num Skip Samples $\geq 08 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 3, (3\text{us} * F_s)]$ samples</td></tr></table>	Chirp start freq. (60GHz devices)	c_DfeFirSel	Skip Samples Recommended	[57.5, 64] GHz	0 (long filter)	Num Skip Samples $\geq 12 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 4, (2\text{us} * F_s)]$ samples	1 (short filter)	Num Skip Samples $\geq 08 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 3, (2\text{us} * F_s)]$ samples	Otherwise	0 (long filter)	Num Skip Samples $\geq 12 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 4, (3\text{us} * F_s)]$ samples	1 (short filter)	Num Skip Samples $\geq 08 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 3, (3\text{us} * F_s)]$ samples		
Chirp start freq. (60GHz devices)	c_DfeFirSel	Skip Samples Recommended														
[57.5, 64] GHz	0 (long filter)	Num Skip Samples $\geq 12 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 4, (2\text{us} * F_s)]$ samples														
	1 (short filter)	Num Skip Samples $\geq 08 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 3, (2\text{us} * F_s)]$ samples														
Otherwise	0 (long filter)	Num Skip Samples $\geq 12 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 4, (3\text{us} * F_s)]$ samples														
	1 (short filter)	Num Skip Samples $\geq 08 + \max [(chirpTxStartTime + 1.5\text{us}) * F_s + 3, (3\text{us} * F_s)]$ samples														
<i>chirpRampEndTime</i>	$\geq \frac{(chirpAdcSkipSamples + \text{numOfAdcSamples} - 3)}{\text{Samplingrate (Mpsps)}}$															
<i>chirpIdleTime constraint 1</i>	$\geq 4.1\text{us} - chirpTxStartTime$ <i>with inter chirp power save disabled</i>															
<i>chirpIdleTime constraint 2</i>	$\geq 3.1\text{us}$															
<i>chirpIdleTime constraint 3</i>	$\geq 1.21\text{us} + \frac{\text{Slope}}{1400 \text{ MHz/us}} \times chirpRampEndTime$															
<i>chirpIdleTime constraint 4</i>	$\geq \frac{(chirpAdcSkipSamples + \text{numOfAdcSamples} + 4)}{\text{Samplingrate (Mpsps)}} - chirpRampEndTime$															
<i>Minimum Burst idle time</i>	$\geq 135\text{us}$ without APLL frequency shift and $\geq 136\text{us}$ with APLL frequency shift and															
<i>Minimum Frame idle time</i>	$\geq 54\text{us}$															

3.2 Detection Layer Parameters

In the following subsections, the detection layer configuration parameters summarized in Table 1 are detailed. Similar to Section 3.1, each table presents the data type and description of each parameter along with the suggested values in an example scenario. Recall that the data types are only classified as ‘int’ for integer numbers and ‘float’ for floating-point numbers. This document does not cover the actual byte size of the parameters (uint8, int32, etc.). Instead, the supported values or valid ranges in each parameter are given in the corresponding section. For more details Please refer to *SDK_INSTALL_PATH /README_FIRST_xWRL684x.html/Examples/MmwaveDemos/* in SDK [1].

3.2.1 Processing Chain Configuration

The following command configures the parameters of the detection layer processing chain.

aoaProcCfg 64 64			
Parameter	Type	Value	Description
azimuthFftSize	int	64	Azimuth FFT size. Suggested to set as power of 2. Refer to the note below ¹ for the details.
elevationFftSize	int	64	Elevation FFT size. Suggested to set as power of 2. Refer to the note below ¹ for the details.

¹**azimuthFftSize and elevationFftSize:** These parameters define the azimuth and elevation FFT sizes in the angle estimation steps. Both parameters, which affect the search granularity (i.e., the inter-bin resolution) in the angle domain, have a significant impact on the processing load (also affects the power consumption). Hence, both parameters should be carefully optimized for the desired use case. Depending on the use case restrictions (power consumption, frame rate, etc.), small azimuth and elevation FFT sizes may be required. This will create coarse angle steps in the azimuth-elevation grid.

3.2.2 Peak Detection Configuration

In the mmWave processing chain, the CFAR DPU implements CFAR object detection from the Range/Doppler detection matrix using hardware accelerator (HWA). CFAR detection is performed in both Range and Doppler domains. The DPU exports the list of objects that are detected both in range and in Doppler domains. The following commands configures the CFAR in range and doppler domains.

cfarProcCfg 0 2 8 4 3 0 9.0 0			
Parameter	Type	Value	Description
procDirection	int	0	CFAR direction selection 0 - CFAR params in range direction 1 - CFAR params in doppler direction
averageMode	int	2	CFAR averaging mode selection:

			0: CFAR-CA. 1: CFAR-CAGO. 2: CFAR-CASO. Suggested to set 2.
winLen	int	8	One-sided noise averaging window length (in samples) of range-CFAR. Suggested to set as power of 2.
guardLen	int	4	One-sided guard length (in samples) of range-CFAR.
noiseDiv	int	3	Cumulative noise sum divisor expressed as a shift. Sum of noise samples is divided by 2^{noiseDiv} . Should be set as $\log_2(\text{winLen})$.
cyclicMode	int	0	Cyclic mode or wrapped around mode: 0: Disabled. 1: Enabled.
thresholdScale	float	9.0	Threshold factor of range-CFAR in dB scale ($20\log_{10}$).
peakGroupingEn	int	0	Peak grouping (in range domain) enable/disable flag: 0: Disabled. 1: Enabled.

cfarProcCfg 1 2 4 2 2 1 9.0 0			
Parameter	Type	Value	Description
procDirection	int	1	CFAR direction selection 0 - CFAR params in range direction 1 - CFAR params in doppler direction
averageMode	int	2	CFAR averaging mode selection: 0: CFAR-CA. 1: CFAR-CAGO. 2: CFAR-CASO. Suggested to set 2. Refer to the note below ¹ for the details.
winLen	int	4	One-sided noise averaging window length (in samples) of range-CFAR. Suggested to set as power of 2. Refer to the note below ² for the details.
guardLen	int	2	One-sided guard length (in samples) of range-CFAR. Refer to the note below ³ for the details.
noiseDiv	int	2	Cumulative noise sum divisor expressed as a shift. Sum of noise samples is divided by 2^{noiseDiv} . Should be set as $\log_2(\text{winLen})$. Refer to the note below ⁴ for the details.
cyclicMode	int	1	Cyclic mode or wrapped around mode: 0: Disabled. 1: Enabled. Refer to the note below ⁵ for the details.

thresholdScale	float	9.0	Threshold factor of range-CFAR in dB scale ($20\log_{10}$). Refer to the note below ⁷ for the details.
peakGroupingEn	int	0	Peak grouping (in range domain) enable/disable flag: 0: Disabled. 1: Enabled. Refer to the note below ⁶ for the details.

The following parameter configures the HWA CFAR engine to decide the version of the CFAR-CA method performed in range domain per angle bin.

- **¹averageMode**: This field configures the mode of the CFAR method in any of the following options:
 - CFAR CA (cell averaging): The average of both left and right reference windows is used as noise estimation for CFAR detection.
 - CFAR CAGO (cell averaging with greater of selection): The greater average of the left and right reference windows are used as noise estimation for the CFAR detection.
 - CFAR CASO (cell averaging with smaller of selection): The smaller average of the left and right reference windows is used as noise estimation for the CFAR detection.

The following CFAR window parameters in the range/doppler domain, which are used to configure the HWA CFAR engine, define a sliding window to calculate the local noise floor to be compared with the cell under test (CUT).

- **²winLen**: This field is the CFAR reference window length (in terms of the number of samples) for the range/doppler domain CFARs. The given value is empirically the best choice, and the user may want to use it as a default setting.
- **³guardLen**: This field is the CFAR guard window size (in terms of the number of samples) in the range/doppler domain. When calculating the detection threshold for the CUT, the left and right **guardLen** of samples will be excluded from noise accumulation. For example, if we have a target with an area of 0.5m^2 reflecting radar energy, we do not want samples in those areas being counted as noise samples to raise the detection threshold. This way, there will be a richer point cloud detection out of the CFAR step. The user should adjust the setting based on the chirp configuration and derived inter-bin resolution, as well as the typical target size within the scene. These settings have already been adjusted for the desired effect on people as intended targets with the provided chirp configuration. It is also important to make sure that $2 \times (\text{winLen} + \text{guardLen}) < \text{numRangeBins}$ and $2 \times (\text{winLen} + \text{guardLen}) < \text{numDopplerBins}$ conditions are satisfied.

The following parameters configure the implementation in HWA CFAR engine.

- **⁴noiseDiv**: This parameter specifies the division factor with which the noise sum calculated from the left and right noise windows are divided in order to get the final surrounding noise average value. The division factor is equal to 2^{noiseDiv} . Therefore, only powers-of-2 division is possible, even

though the number of samples specified in `winLen` is not restricted to powers of 2. The surrounding noise average value obtained after the division is multiplied with `thresholdScale` (in linear scale) to determine the final threshold used to compare the cell under test for detection.

- **`5cyclicMode`**: This parameter specifies whether the CFAR-CA detector needs to work in cyclic mode or in non-cyclic mode. When this field is 0, the CFAR detector works in non-cyclic mode, and when it is 1, it works in cyclic mode. These two modes are different in the way the samples at the edges are handled. In cyclic mode, the CFAR-CA detector needs to wrap around the edges in a circular manner.
- **`6peakGroupingEn`**: This parameter specifies whether peak grouping should be enabled. When this field is 0, peak grouping is disabled, which means that a peak is declared as detected as long as the cell under test exceeds the threshold. On the other hand, if this field is 1, then a peak is declared as detected only if the cell under test exceeds the threshold, as well as, if the cell under test exceeds the two neighboring cells to its immediate left and right (local maximum).

The following CFAR threshold parameter is defined as the ratio of the local noise floor to the CUT, above which a detected point is declared. The lower the thresholds, the more false detections will occur on random noise, object sidelobes, and other spurious environmental effects. The higher the thresholds, the fewer points from the objects of interest are detected.

- **`7thresholdScale`**: This is the relative threshold for CFAR. After noise power (`noisePower`) is calculated based on the `winLen` and `guardLen`, the final detection threshold will be calculated by (`noisePower` × `thresholdScale`), where `noisePower` is the estimated noise power in the reference windows. If the power of CUT is greater than the threshold, it will be declared as a detected point. Note that the scale in this parameter is configured in dB scale ($20\log_{10}$).

This threshold setting impacts detection performance significantly. The lower the threshold, the lower confident detections may show up. The user would see more noisy points around the target and more multipath ghost targets forming between the real target and strong reflectors such as metal beams and walls. On the other hand, if the threshold is too high, the user will lose a detected point that potentially carries valuable information about the target. Loss of this information would affect the performance of the final decision in the ROI, which rely heavily on the statistics/distribution of the range/angle/SNR information of detected points from a target. The suggested values are the best empirical numbers TI obtained, but the user might need to adjust them for their customized setup.

3.2.3 Region of Interest Configuration

The following command defines the sensor azimuth and elevation field-of-views (FOVs). The FOV values in this command define the total angular extent observed at both sides of the sensor. The detected points outside of this angular FOV interval are not exported in the detection list.

aoaFovCfg -60 60 -60 60			
Parameter	Type	Value	Description
minAzimuthDeg	int	-60	Minimum azimuth angle (in degrees) that specifies the start of field of view.
maxAzimuthDeg	int	60	Maximum azimuth angle (in degrees) that specifies the end of field of view.
minElevationDeg	int	-60	Minimum elevation angle (in degrees) that specifies the start of field of view.
maxElevationDeg	int	60	Maximum elevation angle (in degrees) that specifies the end of field of view.

The following commands configures the sensor boundary in the range and doppler domains. The values in this command define the total extent observed by the sensor in range and doppler. The detected points outside of this interval are not exported in the detection list.

cfarFovCfg 0 0.25 9.0			
Parameter	Type	Value	Description
procDirection	int	0	CFAR direction selection 0 - CFAR params in range direction 1 - CFAR params in doppler direction
minMeters or m/sec	float	0.1	Minimum range/velocity of exported detected points Refer to the note below ¹ for the details.
maxMeters or m/sec	float	9.0	Maximum range/velocity of exported detected points Refer to the note below ² for the details.

cfarFovCfg 1 -20.16 20.16			
Parameter	Type	Value	Description
procDirection	int	1	CFAR direction selection 0 - CFAR params in range direction 1 - CFAR params in doppler direction
minMeters or m/sec	float	-20.16	Minimum range/velocity of exported detected points Refer to the note below ¹ for the details.
maxMeters or m/sec	float	20.16	Maximum range/velocity of exported detected points Refer to the note below ² for the details.

¹**min meters or m/sec:** This parameter is the minimum range/velocity that will be included in the created point cloud. For the example chirp configuration, a value of 0.1m in this field implies that the detected points closer than 10cm within the sensor will be excluded from the point cloud. Discarding these range

bins also reduces the throughput of the sensor. If the end application has no interest in detecting the close-by range, these bins can be discarded to save resources.

²max meters or m/sec: This parameter is the maximum range/velocity that will be included in the created point cloud. Depending on the chirp configuration and derived maximum unambiguous range/velocity, this setting dictates the range/velocity furthest away from the sensor (typically the range boundary of the scene) that will be included in the detection result. For the example chirp configuration, a value of 10m in this field implies that distances beyond 10m will not have any detection in the point cloud. Discarding these range bins also reduces the throughput of the sensor. If the end application has no interest in detecting the far range, these bins can be discarded to save resources.

3.2.4 Clutter Removal Configuration

The following parameter enable/disable the clutter removal. The static clutter removal step by zeroing the first Doppler bin is applied to remove the purely static objects, such as chairs and tables, from the scene and leave only the signals backscattered from the moving objects. It is important to note that the clutter removal approach implemented by zeroing the first Doppler bin can only work if the total number of chirp samples in the Doppler processing block is a power of 2, the Doppler FFT size is the same as the number of chirps, and no windowing is used before the Doppler FFT. In this case, there will be no zero paddings in the Doppler FFT processing on the HWA. Hence, the zero Doppler bin will correspond to the average across chirps, which can be used as a static clutter estimation. This parameter is included in the cfg file for debugging or when analyzing the reflections from the static scenes.

clutterRemoval 1			
Parameter	Type	Value	Description
enabled	int	1	Configure static clutter removal: 0: Disabled. 1: Enabled.

3.2.5 Antenna Pattern Configuration

The mmWave demo can support different antenna configurations. Antenna geometry, range bias and phase compensation can be defined using either the antGeometryBoard CLI command or all four of the antGeometryTx, antGeometryRx, antGeometryDist and compRangeBiasAndRxChanPhase CLI commands. The antenna spacing, antenna indices and the virtual antenna array are shown in Figure below. The right-most figure shows the virtual antennas. For calculating the TX/RX indices(row,col) for antGeometryTx/ antGeometryRx, left bottom antenna is considered at origin and lamda/2 is considered for 1 unit in co-ordinate system.

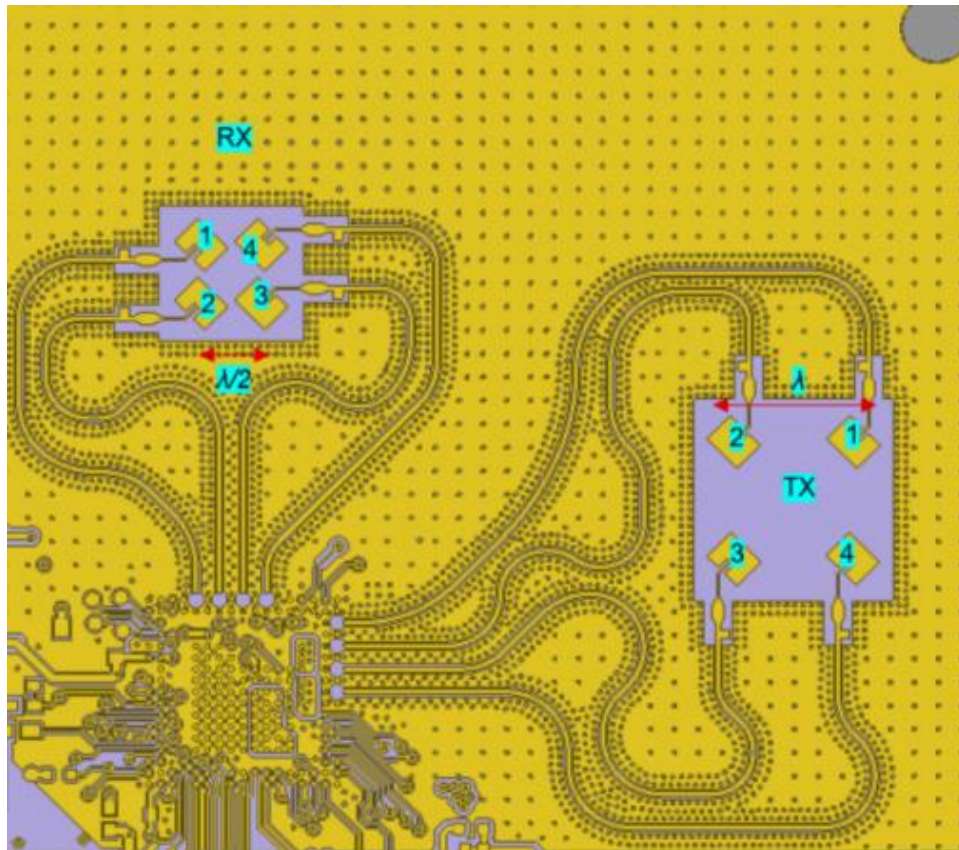
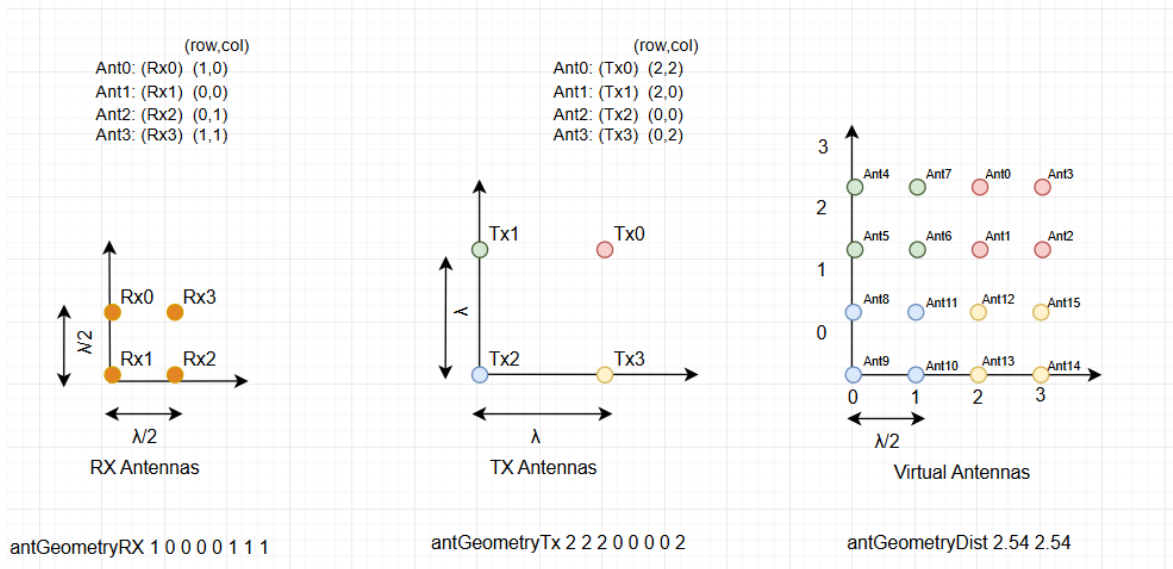


Figure 5: Virtual antenna pattern and the corresponding antenna index for xWRL6844 EVM.

antGeometryBoard xWRL6844EVM			
Parameter	Type	Value	Description
boardName	string	xWRL6844EVM	Name of the EVM board

antGeometryTx 2 2 2 0 0 0 0 2			
Parameter	Type	Value	Description
txAnt1IdxRow	int	2	Row index of TX antenna 1.
txAnt1IdxCol	int	2	Column index of TX antenna 1.
txAnt2IdxRow	int	2	Row index of TX antenna 2.
txAnt 2IdxCol	int	0	Column index of TX antenna 2.
txAnt3IdxRow	int	0	Row index of TX antenna 3.
txAnt 3IdxCol	int	0	Column index of TX antenna 3.
txAnt4IdxRow	int	0	Row index of TX antenna 4.
txAnt 4IdxCol	int	2	Column index of TX antenna 4.

antGeometryRx 1 0 0 0 0 1 1 1			
Parameter	Type	Value	Description
rxAnt1IdxRow	int	1	Row index of RX antenna 1.
rxAnt1IdxCol	int	0	Column index of RX antenna 1.
rxAnt2IdxRow	int	0	Row index of RX antenna 2.
rxAnt 2IdxCol	int	0	Column index of RX antenna 2.
rxAnt3IdxRow	int	0	Row index of RX antenna 3.
rxAnt 3IdxCol	int	1	Column index of RX antenna 3.
rxAnt4IdxRow	int	1	Row index of RX antenna 4.
rxAnt 4IdxCol	int	1	Column index of RX antenna 4.

antGeometryDist 2.54 2.54			
Parameter	Type	Value	Description
antDistX	float	2.54	Antenna spacing in X dimension (i.e., between azimuth columns) in mm.
antDistZ	float	2.54	Antenna spacing in Z dimension (i.e., between elevation rows) in mm.

3.2.6 Range Bias and Phase Compensation Configuration

This set of range and phase compensation parameters can be derived using a corner reflector at sensor boresight (at about a few meters). If the user decides not to compensate for the bias, the default values in the following table can be set in the configuration file.

compRangeBiasAndRxChanPhase 0.0 -1 0 -1 0 -1 0 -1 0 1 0 1 0 1 0 -1 0 -1 0 -1 0 -1 0 1 0 1 0 1 0			
Parameter	Type	Value	Description
rangeBias	float	0.0	Range bias common for all antennas. Not used in the current SDK.
ant1PhaseReal	float	-1	Phase compensation factor (real) of the 1st virtual antenna.
ant1PhaseImag	float	0	Phase compensation factor (imaginary) of the 1st virtual antenna.

ant2PhaseReal	float	-1	Phase compensation factor (real) of the 2nd virtual antenna.
ant2PhaseImag	float	0	Phase compensation factor (imaginary) of the 2nd virtual antenna.
ant3PhaseReal	float	-1	Phase compensation factor (real) of the 3rd virtual antenna.
ant3PhaseImag	float	0	Phase compensation factor (imaginary) of the 3rd virtual antenna.
ant4PhaseReal	float	-1	Phase compensation factor (real) of the 4th virtual antenna.
ant4PhaseImag	float	0	Phase compensation factor (imaginary) of the 4th virtual antenna.
ant5PhaseReal	float	1	Phase compensation factor (real) of the 5th virtual antenna.
ant5PhaseImag	float	0	Phase compensation factor (imaginary) of the 5th virtual antenna.
ant6PhaseReal	float	1	Phase compensation factor (real) of the 6th virtual antenna.
ant6PhaseImag	float	0	Phase compensation factor (imaginary) of the 6th virtual antenna.
ant7PhaseReal	float	1	Phase compensation factor (real) of the 7th virtual antenna.
ant7PhaseImag	float	0	Phase compensation factor (imaginary) of the 7th virtual antenna.
ant8PhaseReal	float	1	Phase compensation factor (real) of the 8th virtual antenna.
ant8PhaseImag	float	0	Phase compensation factor (imaginary) of the 8th virtual antenna.
ant9PhaseReal	float	-1	Phase compensation factor (real) of the 9th virtual antenna.
ant9PhaseImag	float	0	Phase compensation factor (imaginary) of the 9th virtual antenna.
ant10PhaseReal	float	-1	Phase compensation factor (real) of the 10th virtual antenna.
ant10PhaseImag	float	0	Phase compensation factor (imaginary) of the 10th virtual antenna.
ant11PhaseReal	float	-1	Phase compensation factor (real) of the 11th virtual antenna.
ant11PhaseImag	float	0	Phase compensation factor (imaginary) of the 11th virtual antenna.
ant12PhaseReal	float	-1	Phase compensation factor (real) of the 12th virtual antenna.
ant12PhaseImag	float	0	Phase compensation factor (imaginary) of the 12th virtual antenna.
ant13PhaseImag	float	1	Phase compensation factor (imaginary) of the 13th virtual antenna.
ant13PhaseImag	float	0	Phase compensation factor (imaginary) of the 13rd virtual antenna.
ant14PhaseReal	float	1	Phase compensation factor (real) of the 14th virtual antenna.
ant14PhaseImag	float	0	Phase compensation factor (imaginary) of the 14th virtual antenna.
ant15PhaseReal	float	1	Phase compensation factor (real) of the 15th virtual antenna.
ant15PhaseImag	float	0	Phase compensation factor (imaginary) of the 15th virtual antenna.
ant16PhaseReal	float	1	Phase compensation factor (real) of the 16th virtual antenna.
ant16PhaseImag	float	0	Phase compensation factor (imaginary) of the 16th virtual antenna.

It is also important to note that if the sensor is not calibrated and the default calibration coefficients are used in the cfg file (i.e., all 1s in real and all 0s in imaginary parts by default), the phase rotations at each transceiver should also be considered. While some of the antenna elements of the xWRL6844 EVM are fed from the same direction, some of them are fed from opposite direction (e.g., RX1 and RX4 or TX1 and TX4). Hence, the phase rotation for each virtual element is either 1 or -1 based on the feeding direction, by default. However, in the RF front-end of the xWRL6844 sensor, some virtual elements also have a phase rotation of 180 degrees (i.e., -1). As seen in the table above, the corresponding phase compensation factors are multiplied with -1 if there is a phase rotation in the corresponding antenna element.

It is recommended to calibrate the sensor using a corner reflector and enabling the following command and replace these parameters with the board-specific calibration coefficients.

The procedure is implemented by the function `mmwDemo_rangeBiasRxChPhaseMeasure()`. It is called in the processing chain after the range processing DPU. The input to the procedure is a radar cube. The measurement procedure first calculates the range profile as the sum of the magnitude squares across all

antennas on the range FFTs. Then it searches for the peak in the zone $X - D/2 \leq range \leq X + D/2$ where $D = \text{<searchWindow>}$ and $X = \text{<targetDistance>}$.

The procedure then estimates the peak position using the three point parabolic interpolation and the range bias as the difference between the peak position and the configured target distance X .

The Rx channel compensation coefficients are calculated according to following equations.

From the radar cube stored as a complex16 3D array $X[range][chirp][antenna]$, the received symbols are extracted at the range index i_{peak} of the peak position corresponding to the target as

$$x(i) = X(i_{peak}, 0, i) \quad i = 0, \dots, N_{antennas} - 1$$

The coefficients are calculated as

$$c(i) = \frac{x^*(i)}{|x(i)|^2} x_{min}$$

where

$$x_{min} = \min_i \{|x_i|\}$$

The coefficients are sent per frame to the host within the TLV packet.

Measurement Procedure Steps:

1. Set a strong target like corner reflector at boresight at desired distance and measure precisely the distance from the sensor.
2. Create the CLI configuration file for the measurement procedure with
 - a. the **<enabled>** flag to 1 in CLI command **measureRangeBiasAndRxChanPhase**,
 - b. all antennas enabled,
 - c. TDM MIMO mode selected,
3. Run the target demo. The target sends TLV with the calculated coefficients.
4. Copy one snapshot of the coefficients to clipboard.
5. Paste the copied line into the final CLI configuration file.

Note that when the sensors are calibrated, the phase rotations mentioned will already be compensated in the provided calibration coefficients. Note that the following command is not mandatory for the mmWave demo. If this command is not included in the configuration file, the range bias and phase compensation measurements will be disabled by default.

measureRangeBiasAndRxChanPhase 1 2 0.2			
Parameter	Type	Value	Description
enabled	int	1	Enable measurement of the range bias and RX channel gain and phase imperfections:

			0: Disabled. 1: Enabled.
targetDistance	float	2	Distance in meters where the corner reflector is located to be used as test object for measurement. This field is only used when the previous measurement mode is enabled. Note: the corner reflector should be located at the boresight of the sensor when measuring the phase compensation factors.
searchWin	float	0.2	Distance in meters of the search window around targetDistance where the peak will be searched. This field is only used when the previous measurement mode is enabled.

3.2.7 GUI Monitoring Configuration

The following command configures the output data of the mmWave demo streamed over the UART port. Please refer to the demo implementation guide [1] for the details about the UART packet formats of each data output option.

guiMonitor 1 1 0 0 0 1			
Parameter	Type	Value	Description
pointCloud	int	1	Point cloud data transmission enable/disable flag: 0: Disable. 1: Enable point cloud in floating point format, plus side information. Refer to the note below ¹ for the details.
rangeProfile	int	1	Range profile data transmission enable/disable flag: 0: Disable. 1: Enable range profile Refer to the note below ² for the details.
noiseProfile	int	0	Not used in the current SDK, always set to 0.
rangeAzimuthHeatMap	int	0	Not used in the current SDK, always set to 0.
rangeDopplerHeatMap	int	0	Range-doppler heatmap data transmission enable/disable flag: 0: Disable. 1: Enable heatmap Refer to the note below ³ for the details.
statsInfo	int	1	Statistics information (temperature and processing time) transmission enable/disable flag: 0: Disable. 1: Enable. Refer to the note below ⁴ for the details.

¹**pointCloud:** This parameter enables or disables the export of point cloud (x, y, z, Doppler) and point cloud side info (SNR).

²rangeProfile: This parameter enables/disables the export of range profiles. Enabling these outputs has an impact on the throughput. Hence, these parameters should be enabled only in the debug mode with longer frame periods (i.e., slower frame rates).

³rangeDopplerHeat: This parameter enables/disables the export of range-doppler heatmaps. Enabling these outputs has an impact on the throughput. Hence, these parameters should be enabled only in the debug mode with longer frame periods (i.e., slower frame rates).

⁴statsInfo: This parameter enables or disables the export of statistical information about the implementation, such as processing time and the UART transmission time. Both information can be used for benchmarking and performance tuning. This parameter also enables or disables the export of sensor temperature information.

3.3 Debug Related Parameters

The following parameter is used for debugging the target code using captured raw ADC data. When this debug option is enabled, a recorded bin file path can be provided in this command. The generated point cloud by the radar device can then be analyzed in debug mode. For more details about the debug modes please refer to `*SDK_INSTALL_PATH/README_FIRST_xWRL684x.html/Examples/MmwaveDemos/*` in SDK [1].

adcDataSource 0 adc_data_name.bin			
Parameter	Type	Value	Description
enabled	int	0	Enable/disable the target code verification using a captured raw ADC data: 0: Disable 1: Enable
fileName	string	adc_data_name.bin	Full file name including extension and data path. This is the binary file with ADC samples data to be used in debugging.

The following parameter is used to captured raw ADC data using DCA1000 EVM [5].

adcLogging 0			
Parameter	Type	Value	Description
enabled	int	0	ADC data logging enable: 0 - Disable. 1: Enable via DCA.

gpAdcMeasConfig 0 0			
Parameter	Type	Value	Description

channelEnable	int	0	GPADC channels ON/OFF control, 1 bit per channel: bit[0] corresponds to GPADC channel 1, bit[1] corresponds to GPADC channel 2, bit[2] corresponds to GPADC channel 3, bit[3] corresponds to GPADC channel 4.
volPrintsEnable	int	0	GPADC voltage prints. When enabled, the voltages are displayed through UARTB for the GPADC channels configured.

4 Parameter Tuning and Performance

This section details how the configuration parameters discussed in the previous sections can be tuned. This section also describes some commonly scenarios where the user may need to tune the configuration parameters discussed to obtain optimal performance in the processing chain.

NOTE: The mmWave demo currently uses a polling mechanism to detect UART messages, which may introduce unintended spurs in TX spectrum.

4.1 Effect of the Radar Parameters on the Physical Requirements

First, we detail the physical parameters in real-world scenarios and their relations to the radar system parameters discussed in this document. In a real-world scenario, one of the most critical factors to consider is the sensor's ability to distinguish between targets that are very close in either range, velocity, or angle domains, which is typically known as resolution:

- **Range resolution:** The range resolution defines the minimum separation in the range domain at which the radar can distinguish two targets and detect them as separate objects.
- **Velocity resolution:** The velocity resolution is the minimum radial velocity difference between two objects for the radar to detect them as separate objects in the Doppler domain.
- **Angle resolution:** The angle (i.e., azimuth and elevation) resolution defines the minimum separation in the angular domain at which the radar can distinguish two targets and detect them as separate objects.

Additionally, the following parameters are also essential to define the physical extent of the target scene:

- **Maximum unambiguous range:** This parameter specifies the maximum target range that the radar can unambiguously resolve. Targets at ranges beyond the maximum unambiguous range r_{max} cannot be detected because only the range interval $[0, r_{max}]$ is available at the range FFT output.
- **Maximum detection range based on SNR:** The maximum range at which a human can be detected.
- **Maximum unambiguous velocity:** This parameter specifies the maximum magnitude value of the target's radial velocity that the radar can unambiguously resolve. Targets detected at velocities whose magnitude is greater than the maximum unambiguous velocity v_{max} are wrapped into the interval $[-v_{max}, v_{max}]$. In other words, objects moving faster than this value may have incorrect velocity measurements.

4.2 How to Configure the Range

This section lists all the parameters that affect the range measurements and presents the relation of each parameter with the range boundary and resolution in real-world scenarios. The following table establishes the relationship between the system's range requirement and the FMCW chirp waveform parameters.

Commands	Parameters	Tuning Guide
----------	------------	--------------

chirpTimingCfg	chirpRfFreqSlope	This parameter affects the bandwidth. Therefore, it manages both the range resolution and maximum unambiguous range. Increasing the frequency slope improves the range resolution but decreases the maximum unambiguous range. The user must ensure that the occupied bandwidth is within the sensor and region-based regulation (FCC, etc.) limits.
	numOfAdcSamples	This parameter affects the chirp time and bandwidth. When the frequency slope and the sampling rate are constant, it manages the range resolution. Increasing the number of ADC samples results in a wider bandwidth that improves the range resolution. This parameter also affects the valid chirping time and maximum detectable range based on SNR. On the other hand, this parameter affects the radar cube size and required processing power. Changing this parameter may result in the system running out of memory and MIPS.
chirpComnCfg	digOutputSampRate	This parameter affects the chirp time and bandwidth. When the frequency slope and the number of ADC samples are constant, it manages both the range resolution and maximum unambiguous range. In this case, increasing the sampling rate results in a narrower bandwidth that degrades the range resolution but increases the maximum unambiguous range. This parameter also affects the valid chirping time and maximum detectable range based on SNR

TI's mmWave sensors can reduce their transmit power by backing off the maximum output power. This can be tuned according to the detection range requirements of different use cases. Configuring the power as too high (i.e., using a backoff value close to 0dB) may cause stronger multipath reflections and higher false alarms. On the other hand, too low TX power will decrease the detection range performance, which may result in more missed detections.

Commands	Parameters	Tuning Guide
factoryCalibcfg	txBackoff	This parameter affects the maximum detection range and should be configured based on the use case and regulation requirements.

The CFAR-CASO algorithm is applied across the range/doppler domain for peak detection. In the CFAR-CASO scheme for a data set in Figure 6, the noise power is estimated as the average power of the reference cells within a configured window around each cell under test. The CFAR-CASO detector assumes that the reference cells do not contain any signals from targets. Therefore, guard cells prevent noise estimation from signals leaking into the reference cells from the target peaks.

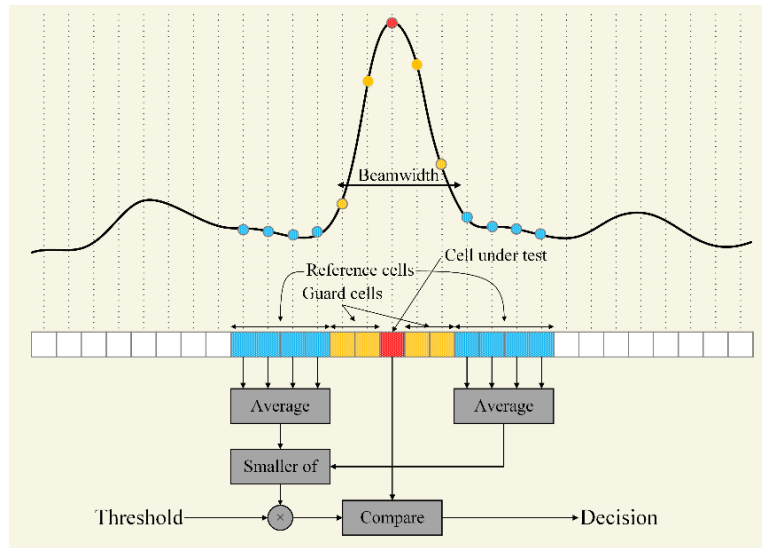


Figure 6: The CFAR-CASO detector estimates the noise variance for the cell under test from the neighboring reference cells.

The reference and guard window sizes in Figure 6 significantly affect detection performance and should be carefully tuned according to the range parameters configured above. As discussed in Section 3.2.2 and illustrated in Figure 6, both window sizes are configured in terms of the number of samples. Hence, depending on the chirp configuration and derived inter-bin range resolution, the user must accordingly tune the following parameters. For example, a higher chirp bandwidth results in a sharper peak in the range FFT output, and therefore, the guard window size can be reduced to improve the noise estimation accuracy. Similarly, when the number of ADC samples or the sampling rate change, the user should adjust the reference and guard cell sizes accordingly.

Commands	Parameters	Tuning Guide
cfarProcCfg	winLen	These parameters are defined in the number of samples. Therefore, when the range resolution and the number of ADC samples are changed, these parameters should be adjusted accordingly.
	guardLen	

4.3 How to Configure the Motion Sensitivity

Optimizing the motion sensitivity (i.e., the velocity resolution) is a crucial task in the mmWave processing chain to be able to distinguish between purely static objects (such as chairs, tables, etc.) and the people who remain stationary or with very slow motion in the scene. A fine-velocity resolution is needed to improve the motion sensitivity. On the other hand, when the velocity resolution is finer, the processing chain becomes more sensitive to undesirable motion artifacts such as sensor vibration. The user should consider a proper tradeoff between detection and false alarm performance according to the target environment.

As presented in Section 0, the time between successive chirps on the same virtual antenna sets the maximum unambiguous velocity that can be measured. Additionally, the total chirp observation window dictates the velocity resolution. The inter-chirp time affects both of these parameters. This section provides a tuning approach for the system's velocity requirements configuring the related radar parameters.

Commands	Parameters	Tuning Guide
frameCfg	numOfChirpsInBurst burstPeriodicity numOfBurstsInFrame	These parameters affect the total chirping window in a single frame. Increasing the number of total chirps improves the velocity resolution but also reduces the total available processing time budget within the frame period. Hence, the user must ensure that the total chirping time and the required processing time can fit into a single frame period. On the other hand, these parameters affect the radar cube size, required processing power, and average power consumption.
	framePeriodicity	The system performance will degrade when detecting the motions (i.e., when too much dynamicity occurs in larger inter-frame durations).
chirpTimingCfg	chirpIdleTime	These parameters affect the inter-chirp time, hence both the velocity resolution and maximum unambiguous velocity. When the number of chirps per frame is the same, increasing the inter-chirp time improves the velocity resolution but decreases the maximum unambiguous velocity. Therefore, the processing chain becomes more sensitive to motion. It is important to note that increasing the inter-chirp time through these parameters increases the total chirping window and reduces the total available processing time budget within the frame period. Hence, the user must ensure that the total chirping time and the required processing time can fit into a single frame period. These parameters also affect the average power consumption.
chirpComnCfg	chirpRampEndTime	

4.4 How to Configure the Angular Accuracy

Based on the sensor mounting option and the required target scene boundary, the user can adjust these angular limits accordingly. However, it is important to emphasize that the user should consider the antenna radiation patterns' physical limitations when configuring these parameters.

The angle resolution described in Section 0 is constrained by the total number of antenna elements used in the sensor array. As discussed in Section 3.1.2, the xWRL6844 is a single-chip radar device with 4 RX and 4 TX. Despite the physical limitations in the angle resolution imposed by the antenna pattern, the angle estimation accuracy can be improved by increasing the angular search grid in the FFT-based angle

estimation step. This section provides a tuning approach for the angular accuracy requirements by configuring the related radar parameters.

Commands	Parameters	Tuning Guide
aoaProcCfg	azimuthFftSize elevationFftSize	These parameters configure the angle search resolution in both azimuth and elevation domains. A larger value will improve the estimated azimuth/elevation accuracy but also increase the total processing time. The user should make a tradeoff and adjust the angle FFT sizes according to the memory, computation power, and average power consumption limits. Changing this parameter may result in the system running out of memory and MIPS. To fit the required processing time into a single frame period, the user must configure the frame period.

5 References

- [1] MMWAVE-L-SDK: xWRL6844 software development kit for low-power mmWave radar sensors, Texas Instruments. [Online]. Available: <https://www.ti.com/tool/MMWAVE-L-SDK>
- [2] People counting demo: 'Tracking radar targets with multiple reflection points', Texas Instruments. [Online]. Available: <https://dev.ti.com/tirex/explore>
- [3] Vivek Dham, "Programming chirp parameters in TI devices," Texas Instruments, Application Report, SWRA553A, February 2020.
- [4] Radar hardware accelerator user guide, SWRU527A, March 2018.
- [5] DCA1000EVM: Real-time data-capture adapter for radar sensing evaluation module, Texas Instruments. [Online]. Available: <https://www.ti.com/tool/DCA1000EVM>